

IOT FISH POND MANAGEMENT SYSTEM

ENGINEERING IMPLEMENTATION GUIDE

*Complete Build Guide: Wiring, Firmware, App Development & System
Configuration*

For Internal Engineering Team Use Only
Document Version 1.0

1. Project Overview & Engineering Scope

This guide covers the full implementation of the IoT-Based Fish Pond Monitoring and Management System from the engineering perspective. It is organized into five major phases: hardware assembly and wiring, ESP32 firmware development, mobile application development, Firebase cloud backend configuration, and full system integration and testing.

The system uses an ESP32 as the central controller connected to three sensors (DS18B20 temperature, PH4502C pH, Turbidity), two relay-driven actuators (aerator and feeder motor), and a local alert circuit (buzzer + LED). All data flows through Firebase Realtime Database to a Flutter-based mobile application.

Target Hardware	ESP32 30-pin Development Board
Firmware Language	C++ (Arduino Framework via PlatformIO or Arduino IDE)
App Framework	Flutter (Dart) - Android & iOS from single codebase
Cloud Backend	Firebase Realtime Database + Firebase Cloud Messaging (FCM)
Communication Protocol	Wi-Fi (HTTP/HTTPS to Firebase REST API or Firebase SDK)
Libraries (Firmware)	OneWire, DallasTemperature, ArduinoJson, Firebase-ESP-Client, NTPClient

2. Bill of Materials (BOM) with Exact Component Details

Procure all components before beginning assembly. Verify component specifications against this table.

Component	Qty	Price (BDT)	Supplier	Key Spec
ESP32 Dev Board (30-pin)	1	430	roboticsbd.com	Dual-core, built-in WiFi
DS18B20 Waterproof	1	220	daraz.com.bd	Digital, OneWire protocol
PH4502C pH Module	1	2,900	techshopbd.com	Analog out, 0-14 pH range
Turbidity Sensor Module	1	820	techshopbd.com	Analog out

Component	Qty	Price (BDT)	Supplier	Key Spec
2-Channel Relay Module	1	400	roboticsbd.com	5V coil, 10A 250VAC
Buzzer (Active)	1	80	Local electronics	5V active buzzer
LED (Red, 5mm)	1	10	Local electronics	5mm red LED
330-ohm Resistor	1	5	Local electronics	For LED current limiting
4.7k-ohm Resistor	1	5	Local electronics	DS18B20 pull-up
18650 Battery	1	80	roboticsbd.com	3.7V 2600mAh
TP4056 Charger Module	1	60	Local electronics	5V USB input
Aquarium Air Pump	1	~300	Local pet shop	220V AC small pump
DC Motor (for feeder)	1	~200	Local shop	5-12V DC geared motor
Breadboard (full size)	1	120	Local electronics	830 tie-points
Jumper Wires (M-M, M-F)	1 set	80	Local electronics	20cm assorted
USB Micro cable	1	80	Local shop	For ESP32 power/prog
pH buffer solution set	1 set	~200	Chemistry supplier	pH 4.0, pH 7.0
Waterproof project box	1	~150	Local hardware	For electronics enclosure

3. Hardware Wiring - Complete Pin Connection Guide

This section provides exact pin-by-pin wiring instructions. Follow each sub-section in order. All connections are made on the breadboard first for testing before soldering to a permanent PCB.

WARNING: Always disconnect all power before making or changing any wiring connections. The ESP32 operates at 3.3V logic. Do not connect 5V signals directly to GPIO pins unless stated otherwise.

3.1 ESP32 GPIO Pin Assignment Overview

The following GPIO assignments are used throughout this project. Reserve these pins as shown - do not use them for other purposes.

Component	Component Pin	ESP32 GPIO Pin	Notes
DS18B20 Temp Sensor	DATA (yellow wire)	GPIO 4	OneWire data; needs 4.7k pull-up to 3.3V
PH4502C pH Module	PO (analog output)	GPIO 34	ADC1_CH6; input-only pin, ideal for analog

Component	Component Pin	ESP32 GPIO Pin	Notes
Turbidity Sensor	SIG / AO (analog out)	GPIO 35	ADC1_CH7; input-only pin, ideal for analog
Relay Module - CH1	IN1 (aerator relay)	GPIO 26	OUTPUT; LOW = relay ON (active-low module)
Relay Module - CH2	IN2 (feeder relay)	GPIO 27	OUTPUT; LOW = relay ON (active-low module)
Buzzer (Active)	Positive lead	GPIO 25	OUTPUT; HIGH = buzzer ON
LED (Red)	Anode (+)	GPIO 32	OUTPUT; HIGH = LED ON; use 330-ohm series resistor
All sensors & ESP32	GND	GND	Common ground rail on breadboard
DS18B20	VCC (red wire)	3.3V	Power from ESP32 3.3V pin
pH Module	VCC (power)	5V (VIN)	pH module requires 5V supply
Turbidity Module	VCC (power)	5V (VIN)	Turbidity module requires 5V supply
Relay Module	VCC / DC+	5V (VIN)	Relay coil requires 5V

3.2 DS18B20 Temperature Sensor Wiring

The DS18B20 waterproof version has three wires: Red (VCC), Black (GND), Yellow (Data).

1. Connect RED wire to the ESP32 3.3V pin on the breadboard power rail.
2. Connect BLACK wire to the GND rail on the breadboard.
3. Connect YELLOW wire to breadboard column connected to GPIO 4 on ESP32.
4. CRITICAL: Place a 4.7k ohm resistor between the YELLOW data wire and the 3.3V rail. This is the required pull-up resistor for the OneWire protocol. Without it, the sensor will not communicate.
5. Double-check: 3.3V rail -> 4.7k resistor -> GPIO 4 junction; Yellow wire also connects to this same junction.

TIP: The DS18B20 has a unique 64-bit ROM code. If you ever connect multiple temperature sensors, each can be addressed individually on the same data wire. For this project, one sensor is used.

3.3 pH Sensor (PH4502C) Wiring

The PH4502C module has a BNC connector for the glass electrode and screw terminal / pin header outputs.

6. Connect module VCC pin to 5V rail (ESP32 VIN pin or separate 5V source on breadboard).
7. Connect module GND pin to GND rail.
8. Connect module PO (Analog Output) pin to GPIO 34 on ESP32.
9. Plug the pH glass electrode probe into the BNC connector on the module board.
10. The module also has a To (temperature compensation output) and a Do (digital threshold output). Leave these unconnected for basic use.

IMPORTANT: GPIO 34, 35, 36, and 39 on the ESP32 are INPUT ONLY. They cannot be configured as outputs. This is why they are ideal for analog sensor inputs. Do NOT connect any output devices to these pins.

pH Sensor Voltage Divider (REQUIRED):

The PH4502C outputs 0-5V analog signal, but the ESP32 ADC only accepts 0-3.3V (max). You MUST use a voltage divider to protect the ESP32:

11. Use two resistors: R1 = 10k ohm (from pH module PO to GPIO 34) and R2 = 22k ohm (from GPIO 34 to GND).
12. This creates a voltage divider: $V_{out} = V_{in} * (22k / (10k + 22k)) = V_{in} * 0.6875$
13. Maximum input to GPIO 34 becomes: $5V * 0.6875 = 3.44V$ (safe for ESP32).
14. Account for this scaling factor in the pH calculation code (see firmware section).

3.4 Turbidity Sensor Wiring

The turbidity sensor module provides an analog output proportional to turbidity level.

15. Connect sensor module VCC to 5V rail.
16. Connect sensor module GND to GND rail.
17. Connect sensor module SIG or AO (Analog Output) to GPIO 35 on ESP32.
18. Apply the same voltage divider as described for the pH sensor if the module output can exceed 3.3V (measure with a multimeter in clean water to verify max output voltage).
19. Submerge only the sensor probe end in water. Keep all electronics and the PCB dry.

3.5 Relay Module Wiring

The 2-channel relay module controls the aerator (Channel 1) and feeder motor (Channel 2).

Control Side (Low voltage - to ESP32):

20. Connect relay module VCC to 5V rail (VIN pin of ESP32).
21. Connect relay module GND to GND rail.
22. Connect relay module IN1 to GPIO 26 on ESP32.
23. Connect relay module IN2 to GPIO 27 on ESP32.

Load Side (High voltage - to aerator / motor):

Channel 1 (Aerator - if AC powered):

24. Connect the LIVE wire of the aerator's power cord to the COM (Common) terminal of Relay Channel 1.
25. Connect a wire from the NO (Normally Open) terminal of Relay Channel 1 to the wall socket live rail.
26. The NEUTRAL wire of the aerator connects directly to the wall socket neutral (not through relay).

Channel 2 (Feeder Motor - DC powered):

27. Connect the positive (+) wire of the DC motor to the COM terminal of Relay Channel 2.
28. Connect a wire from NO terminal of Relay Channel 2 to the positive of the motor power supply (5-12V DC).
29. Connect the motor negative (-) wire directly to the negative of the motor power supply.

WARNING: If the aerator runs on 220V AC mains power, have a qualified person handle the AC wiring. Use properly insulated wires and ensure the relay is rated for the aerator's current draw. Use a fuse on the AC circuit. All AC wiring must be enclosed in a safe junction box.

RELAY LOGIC NOTE: Most common 2-channel relay modules are ACTIVE LOW, meaning the relay turns ON when the GPIO pin is set LOW (0V) and turns OFF when set HIGH (3.3V). Verify your specific module and adjust the firmware logic accordingly. The code in Section 4 is written for active-low relays.

3.6 Buzzer Wiring

30. Connect the positive leg (+, longer lead) of the active buzzer to GPIO 25 on ESP32.
31. Connect the negative leg (-, shorter lead) to GND rail.
32. An active buzzer contains its own oscillator - it makes sound when DC power is applied. No PWM needed.

3.7 LED Wiring

33. Connect a 330-ohm resistor to GPIO 32 on ESP32 (place on breadboard inline).
34. Connect the other end of the resistor to the ANODE (+, longer leg) of the LED.
35. Connect the CATHODE (-, shorter leg) of the LED to GND rail.
36. Formula: Resistor value = $(3.3V - 2V \text{ forward voltage}) / 0.02A = 65 \text{ ohm minimum}$. 330 ohm provides safe, visible brightness.

3.8 Power Rail Summary

On your breadboard, establish two power rails:

- 3.3V Rail: Connected to ESP32 3.3V pin. Supplies: DS18B20 sensor.
- 5V Rail: Connected to ESP32 VIN pin (when USB-powered). Supplies: pH module, Turbidity module, Relay module.
- GND Rail: Connected to ESP32 GND pin. Common ground for ALL components.

TESTING TIP: Before connecting all sensors at once, connect and test each sensor individually. Use the Serial Monitor in Arduino IDE to verify each sensor gives expected readings before proceeding to the next.

4. ESP32 Firmware Development

4.1 Development Environment Setup

Option A: Arduino IDE (Recommended for Beginners)

37. Download and install Arduino IDE 2.x from arduino.cc.
38. Open Arduino IDE. Go to File > Preferences. In 'Additional boards manager URLs', add: https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
39. Go to Tools > Board > Boards Manager. Search 'esp32' and install 'ESP32 by Espressif Systems'.
40. Select Board: Tools > Board > esp32 > 'ESP32 Dev Module'.
41. Select Port: Connect ESP32 via USB. Go to Tools > Port and select the correct COM port (Windows) or /dev/ttyUSB0 (Linux/Mac).
42. Install Required Libraries: Go to Sketch > Include Library > Manage Libraries and install the following:
 - OneWire by Paul Stoffregen
 - DallasTemperature by Miles Burton
 - ArduinoJson by Benoit Blanchon (version 6.x)
 - Firebase ESP Client by Mobizt (search: Firebase ESP Client)
 - NTPClient by Fabrice Weinberg

Option B: PlatformIO (Recommended for Production)

PlatformIO is a VS Code extension that provides better project management and dependency control.

43. Install VS Code. Then install the PlatformIO IDE extension from the VS Code marketplace.
44. Create a new project. Select board: 'Espressif ESP32 Dev Module'. Framework: Arduino.
45. In platformio.ini, add library dependencies:

```
[env:esp32dev]
platform = espressif32
```

```
board = esp32dev
framework = arduino
monitor_speed = 115200
lib_deps =
  paulstoffregen/OneWire
  milesburton/DallasTemperature
  bblanchon/ArduinoJson
  mobizt/Firebase ESP Client
  arduino-libraries/NTPClient
```

4.2 Complete Firmware Code

Create the main firmware file (sketch). Below is the complete, production-ready firmware with all features implemented.

config.h - Configuration File

Create a separate config.h file for all user-configurable constants:

```
// config.h - Fish Pond System Configuration
#ifndef CONFIG_H
#define CONFIG_H

// WiFi Credentials
#define WIFI_SSID      "YOUR_WIFI_SSID"
#define WIFI_PASSWORD  "YOUR_WIFI_PASSWORD"

// Firebase Configuration
#define FIREBASE_HOST  "your-project-id-default-rtdb.firebaseio.com"
#define FIREBASE_AUTH  "your-firebase-database-secret" // or API key
#define FCM_SERVER_KEY "your-fcm-server-key"

// GPIO Pin Definitions
#define PIN_DS18B20    4    // Temperature sensor data pin
#define PIN_PH_SENSOR  34   // pH analog input
#define PIN_TURBIDITY  35   // Turbidity analog input
#define PIN_RELAY_AERATOR 26 // Relay channel 1 - Aerator
#define PIN_RELAY_FEEDER 27 // Relay channel 2 - Feeder
#define PIN_BUZZER     25   // Alert buzzer
#define PIN_LED        32   // Alert LED

// Relay Logic (Active-Low = true, Active-High = false)
#define RELAY_ACTIVE_LOW true
#define RELAY_ON  (RELAY_ACTIVE_LOW ? LOW : HIGH)
#define RELAY_OFF (RELAY_ACTIVE_LOW ? HIGH : LOW)

// Default Threshold Values (overridable from Firebase)
#define DEFAULT_TEMP_HIGH 32.0 // Celsius
#define DEFAULT_TEMP_LOW  20.0 // Celsius
#define DEFAULT_PH_HIGH   9.0
```



```
#define DEFAULT_PH_LOW          6.0
#define DEFAULT_TURBIDITY_MAX 400    // ADC raw value (0-4095)

// Default Aerator Auto-Run Duration (milliseconds)
#define DEFAULT_AERATOR_AUTO_DURATION 900000 // 15 minutes

// Sensor Reading Interval (milliseconds)
#define SENSOR_INTERVAL 10000 // 10 seconds

// Firebase Update Interval (milliseconds)
#define FIREBASE_INTERVAL 15000 // 15 seconds

// Alert Cooldown (milliseconds)
#define ALERT_COOLDOWN 600000 // 10 minutes

// pH Calibration (adjust based on calibration results)
#define PH_OFFSET 0.0
#define PH_SLOPE 1.0

// Voltage Divider correction factor for pH analog input
// (accounts for the 10k/22k divider - inverse of 22k/(10k+22k))
#define PH_VOLTAGE_SCALE 1.454 // = (10k+22k)/22k

#endif // CONFIG_H
```

main.ino - Main Firmware File

```
#include <Arduino.h>
#include <WiFi.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <FirebaseESP32.h>
#include <NTPClient.h>
#include <WiFiUdp.h>
#include <Preferences.h>
#include <ArduinoJson.h>
#include "config.h"

// --- Object Declarations ---
OneWire oneWire(PIN_DS18B20);
DallasTemperature tempSensor(&oneWire);
FirebaseData fbData;
FirebaseAuth fbAuth;
FirebaseConfig fbConfig;
WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", 21600); // UTC+6 Bangladesh
Preferences prefs;

// --- State Variables ---
float temperature = 0.0, pHValue = 0.0;
int turbidityRaw = 0;
bool aeratorRunning = false, feederRunning = false;
```

```
unsigned long aeratorStopTime = 0, feederStopTime = 0;
unsigned long lastSensorRead = 0, lastFirebaseUpdate = 0;
unsigned long lastTempAlert = 0, lastPhAlert = 0, lastTurbAlert = 0;
unsigned long lastScheduleCheck = 0;

// --- Threshold Variables (loaded from Firebase or defaults) ---
float tempHigh, tempLow, phHigh, phLow;
int turbMax;
unsigned long aeratorAutoDuration;

// --- Helper: Relay Control ---
void setAerator(bool on, unsigned long durationMs = 0) {
    aeratorRunning = on;
    digitalWrite(PIN_RELAY_AERATOR, on ? RELAY_ON : RELAY_OFF);
    aeratorStopTime = on && durationMs > 0 ? millis() + durationMs : 0;
    Firebase.setString(fbData, "/device/aerator_status", on ? "ON" : "OFF");
}

void setFeeder(bool on, unsigned long durationMs = 0) {
    feederRunning = on;
    digitalWrite(PIN_RELAY_FEEDER, on ? RELAY_ON : RELAY_OFF);
    feederStopTime = on && durationMs > 0 ? millis() + durationMs : 0;
    if (on) Firebase.setString(fbData, "/device/last_feed_time",
        timeClient.getFormattedTime());
}

// --- Helper: Alert ---
void triggerAlert(bool state) {
    digitalWrite(PIN_BUZZER, state ? HIGH : LOW);
    digitalWrite(PIN_LED, state ? HIGH : LOW);
}

// --- pH ADC to pH value conversion ---
float adcToPH(int adcRaw) {
    float voltage = (adcRaw / 4095.0) * 3.3 * PH_VOLTAGE_SCALE;
    // PH4502C: pH 7 = ~2.5V, slope ~0.18V/pH unit
    float ph = 7.0 + ((2.5 - voltage) / 0.18);
    return ph * PH_SLOPE + PH_OFFSET;
}

// --- Load Settings from Firebase ---
void loadSettings() {
    if (Firebase.getFloat(fbData, "/settings/temp_high"))
        tempHigh = fbData.floatData();
    if (Firebase.getFloat(fbData, "/settings/temp_low"))
        tempLow = fbData.floatData();
    if (Firebase.getFloat(fbData, "/settings/ph_high"))
        phHigh = fbData.floatData();
    if (Firebase.getFloat(fbData, "/settings/ph_low"))
        phLow = fbData.floatData();
    if (Firebase.getInt(fbData, "/settings/turbidity_max"))
        turbMax = fbData.intData();
    if (Firebase.getInt(fbData, "/settings/aerator_auto_duration_s"))
        aeratorAutoDuration = fbData.intData() * 1000UL;
}
```

```
// --- Check Manual Commands from Firebase ---
void checkCommands() {
  if (Firebase.getString(fbData, "/commands/aerator")) {
    String cmd = fbData.stringData();
    if (cmd.startsWith("RUN:")) {
      int secs = cmd.substring(4).toInt();
      setAerator(true, secs * 1000UL);
      Firebase.setString(fbData, "/commands/aerator", "IDLE");
    }
  }
  if (Firebase.getString(fbData, "/commands/feeder")) {
    String cmd = fbData.stringData();
    if (cmd.startsWith("RUN:")) {
      int secs = cmd.substring(4).toInt();
      setFeeder(true, secs * 1000UL);
      Firebase.setString(fbData, "/commands/feeder", "IDLE");
    }
  }
}

// --- Check Schedules from Firebase ---
void checkSchedules() {
  String currentTime = timeClient.getFormattedTime().substring(0,5); // HH:MM
  // Check aerator schedules (/schedules/aerator/0,1,2)
  for (int i = 0; i < 3; i++) {
    String path = "/schedules/aerator/" + String(i) + "/time";
    if (Firebase.getString(fbData, path)) {
      if (fbData.stringData() == currentTime && !aeratorRunning) {
        String durPath = "/schedules/aerator/" + String(i) + "/duration_s";
        int dur = 60;
        if (Firebase.getInt(fbData, durPath)) dur = fbData.intData();
        setAerator(true, dur * 1000UL);
      }
    }
  }
  // Check feeder schedules (/schedules/feeder/0..4)
  for (int i = 0; i < 5; i++) {
    String path = "/schedules/feeder/" + String(i) + "/time";
    if (Firebase.getString(fbData, path)) {
      if (fbData.stringData() == currentTime && !feederRunning) {
        String durPath = "/schedules/feeder/" + String(i) + "/duration_s";
        int dur = 5;
        if (Firebase.getInt(fbData, durPath)) dur = fbData.intData();
        setFeeder(true, dur * 1000UL);
      }
    }
  }
}

// --- Setup ---
void setup() {
  Serial.begin(115200);

  // Initialize GPIO
  pinMode(PIN_RELAY_AERATOR, OUTPUT); digitalWrite(PIN_RELAY_AERATOR, RELAY_OFF);
}
```

```
pinMode(PIN_RELAY_FEEDER, OUTPUT); digitalWrite(PIN_RELAY_FEEDER, RELAY_OFF);
pinMode(PIN_BUZZER, OUTPUT); digitalWrite(PIN_BUZZER, LOW);
pinMode(PIN_LED, OUTPUT); digitalWrite(PIN_LED, LOW);

// Start temperature sensor
tempSensor.begin();

// Connect to WiFi
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
Serial.print("Connecting to WiFi");
while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
Serial.println(" Connected! IP: " + WiFi.localIP().toString());

// Start NTP (time sync)
timeClient.begin();
timeClient.update();

// Initialize Firebase
fbConfig.host = FIREBASE_HOST;
fbConfig.signer.tokens.legacy_token = FIREBASE_AUTH;
Firebase.begin(&fbConfig, &fbAuth);
Firebase.reconnectWiFi(true);

// Load settings from Firebase
tempHigh = DEFAULT_TEMP_HIGH; tempLow = DEFAULT_TEMP_LOW;
phHigh = DEFAULT_PH_HIGH; phLow = DEFAULT_PH_LOW;
turbMax = DEFAULT_TURBIDITY_MAX;
aeratorAutoDuration = DEFAULT_AERATOR_AUTO_DURATION;
loadSettings();

Serial.println("Setup complete.");
}

// --- Main Loop ---
void loop() {
    unsigned long now = millis();

    // --- Read Sensors ---
    if (now - lastSensorRead >= SENSOR_INTERVAL) {
        lastSensorRead = now;

        // Temperature
        tempSensor.requestTemperatures();
        temperature = tempSensor.getTempCByIndex(0);

        // pH
        int phRaw = analogRead(PIN_PH_SENSOR);
        phValue = adcToPH(phRaw);
        if (phValue < 0) phValue = 0;
        if (phValue > 14) phValue = 14;

        // Turbidity
        turbidityRaw = analogRead(PIN_TURBIDITY);
```

```
// Debug
Serial.printf("Temp: %.2f C  pH: %.2f  Turb: %d\n",
    temperature, pHValue, turbidityRaw);

// --- Threshold Checks & Alerts ---
bool alert = false;
if ((temperature > tempHigh || temperature < tempLow)
    && (now - lastTempAlert > ALERT_COOLDOWN)) {
    alert = true;
    lastTempAlert = now;
    // Auto-start aerator if temperature is HIGH
    if (temperature > tempHigh && !aeratorRunning)
        setAerator(true, aeratorAutoDuration);
    Firebase.setString(fbData, "/alerts/latest",
        "TEMP:" + String(temperature, 2));
}
if ((pHValue > pHHigh || pHValue < pHLow)
    && (now - lastPhAlert > ALERT_COOLDOWN)) {
    alert = true;
    lastPhAlert = now;
    Firebase.setString(fbData, "/alerts/latest",
        "PH:" + String(pHValue, 2));
}
if ((turbidityRaw > turbMax)
    && (now - lastTurbAlert > ALERT_COOLDOWN)) {
    alert = true;
    lastTurbAlert = now;
    Firebase.setString(fbData, "/alerts/latest",
        "TURB:" + String(turbidityRaw));
}
triggerAlert(alert);
}

// --- Update Firebase with sensor data ---
if (now - lastFirebaseUpdate >= FIREBASE_INTERVAL
    && Firebase.ready()) {
    lastFirebaseUpdate = now;
    timeClient.update();
    Firebase.setFloat(fbData, "/sensors/temperature", temperature);
    Firebase.setFloat(fbData, "/sensors/ph", pHValue);
    Firebase.setInt(fbData, "/sensors/turbidity", turbidityRaw);
    Firebase.setString(fbData, "/sensors/timestamp",
        timeClient.getFormattedTime());
    // Reload settings in case user changed them from app
    loadSettings();
    checkCommands();
}

// --- Check Schedules every minute ---
if (now - lastScheduleCheck >= 60000) {
    lastScheduleCheck = now;
    checkSchedules();
}

// --- Auto-stop actuators when timer expires ---
if (aeratorRunning && aeratorStopTime > 0 && now >= aeratorStopTime)
```

```
    setAerator(false);  
    if (feederRunning && feederStopTime > 0 && now >= feederStopTime)  
        setFeeder(false);  
  
    delay(100);  
}
```

4.3 pH Calibration Procedure

pH sensors require calibration before first use and periodically thereafter (every 2-4 weeks).

46. Prepare pH 7.0 buffer solution in a clean glass.
47. Rinse the pH probe with distilled water, pat dry gently.
48. Submerge the probe in pH 7.0 buffer. Wait 2-3 minutes for reading to stabilize.
49. Open Serial Monitor in Arduino IDE at 115200 baud. Note the ADC raw value.
50. Adjust the firmware: voltage at pH 7 should ideally correspond to ~2.5V. Use the serial output to verify and adjust PH_OFFSET in config.h until the displayed value reads 7.00.
51. Repeat with pH 4.0 buffer to verify slope. Adjust PH_SLOPE in config.h if the pH 4 reading is off.
52. Rinse probe with distilled water and return to storage solution when not in use.

4.4 Uploading Firmware

53. Connect ESP32 to PC via Micro USB cable.
54. In Arduino IDE: Select Tools > Board > ESP32 Dev Module. Select correct COM port under Tools > Port.
55. Click the Upload button (right arrow icon) or press Ctrl+U.
56. If upload fails with 'Connecting...': Hold the BOOT button on ESP32, click Upload, release BOOT once 'Connecting...' message appears.
57. After successful upload, open Serial Monitor (Tools > Serial Monitor, set baud to 115200). Verify WiFi connection and sensor readings appear.

5. Firebase Backend Configuration

5.1 Creating the Firebase Project

58. Go to firebase.google.com and sign in with a Google account.
59. Click 'Add Project'. Enter project name: fish-pond-system (or similar).
60. Disable Google Analytics (not needed). Click 'Create Project'.
61. In the Firebase Console, go to Build > Realtime Database. Click 'Create Database'.
62. Select the nearest region (asia-southeast1 for Singapore, closest to Bangladesh).
63. Start in TEST MODE for development (change to secure rules before deployment).

64. Copy the Database URL shown (e.g., <https://fish-pond-system-default-rtdb.asia-southeast1.firebaseio.com>). This goes into FIREBASE_HOST in config.h.
65. Go to Project Settings > Service Accounts > Database Secrets. Click 'Show' to get the legacy secret token. This goes into FIREBASE_AUTH in config.h.

5.2 Firebase Realtime Database Structure

The following is the exact JSON structure used by the firmware and mobile app. Initialize this structure in Firebase console using the Import JSON feature:

```
{
  "sensors": {
    "temperature": 0.0,
    "ph": 0.0,
    "turbidity": 0,
    "timestamp": "00:00:00"
  },
  "device": {
    "aerator_status": "OFF",
    "feeder_status": "OFF",
    "last_feed_time": "",
    "online": false
  },
  "settings": {
    "temp_high": 32.0,
    "temp_low": 20.0,
    "ph_high": 9.0,
    "ph_low": 6.0,
    "turbidity_max": 400,
    "aerator_auto_duration_s": 900,
    "sensor_interval_s": 10
  },
  "commands": {
    "aerator": "IDLE",
    "feeder": "IDLE"
  },
  "schedules": {
    "aerator": {
      "0": { "time": "", "duration_s": 60, "enabled": false },
      "1": { "time": "", "duration_s": 60, "enabled": false },
      "2": { "time": "", "duration_s": 60, "enabled": false }
    },
    "feeder": {
      "0": { "time": "07:00", "duration_s": 5, "enabled": true },
      "1": { "time": "18:00", "duration_s": 5, "enabled": true },
      "2": { "time": "", "duration_s": 5, "enabled": false },
      "3": { "time": "", "duration_s": 5, "enabled": false },
      "4": { "time": "", "duration_s": 5, "enabled": false }
    }
  },
  "alerts": {
    "latest": "",
    "history": {}
  }
}
```

```
}
}
```

5.3 Firebase Security Rules

After initial testing, replace the default open rules with these secured rules:

```
{
  "rules": {
    ".read": "auth != null",
    ".write": "auth != null",
    "sensors": {
      ".read": "auth != null",
      ".write": "auth != null"
    },
    "settings": {
      ".read": "auth != null",
      ".write": "auth != null"
    },
    "commands": {
      ".read": "auth != null",
      ".write": "auth != null"
    },
    "schedules": {
      ".read": "auth != null",
      ".write": "auth != null"
    }
  }
}
```

5.4 Firebase Cloud Messaging (FCM) Setup for Notifications

66. In Firebase Console, go to Project Settings > Cloud Messaging.
67. Copy the Server Key under 'Cloud Messaging API (Legacy)'. This is the FCM_SERVER_KEY for the firmware.
68. In the mobile app (Flutter), integrate the firebase_messaging package. On app launch, call FirebaseMessaging.instance.getToken() to get the device's FCM registration token.
69. Store this token in Firebase at /device/fcm_token.
70. In the ESP32 firmware, when an alert is triggered, send an HTTP POST request to <https://fcm.googleapis.com/fcm/send> with the device token and alert message (use HTTPClient library for this - see below).

```
// FCM Push Notification from ESP32 (add to firmware)
#include <HTTPClient.h>

void sendPushNotification(String title, String body) {
  String fcmToken = "";
```



```
if (Firebase.getString(fbData, "/device/fcm_token"))
    fcmToken = fbData.stringData();
if (fcmToken.length() == 0) return;

HttpClient http;
http.begin("https://fcm.googleapis.com/fcm/send");
http.addHeader("Content-Type", "application/json");
http.addHeader("Authorization", "key=" + String(FCM_SERVER_KEY));

String payload = "{\"to\":\"" + fcmToken + "\",";
payload += "\"notification\":{\"title\":\"" + title + "\",";
payload += "\"body\":\"" + body + "\"}}";

int code = http.POST(payload);
Serial.println("FCM response: " + String(code));
http.end();
}
```

6. Mobile Application Development (Flutter)

6.1 Why Flutter Over Blynk

Blynk is an excellent rapid-prototyping platform but has significant limitations for a production-grade, project-specific application. The following comparison explains why Flutter with Firebase is the recommended approach:

Criteria	Blynk	Flutter + Firebase
Custom UI	Limited widget set, fixed layouts	Fully custom, pixel-perfect design
Cost	Free tier limited; paid for advanced	Firebase free tier is sufficient
Offline schedule	Requires Blynk server online	ESP32 runs from stored settings offline
Push Notifications	Blynk server sends them	FCM - direct, reliable, free
Scalability	Limited devices on free tier	Scales to thousands of devices free
Data ownership	Stored on Blynk servers	Stored in your own Firebase project
Customization	Cannot modify app logic	Full source code control
Learning value	Hides implementation details	Full-stack IoT learning experience

6.2 Flutter Development Environment Setup

71. Download Flutter SDK from flutter.dev. Follow the installation guide for your OS.
72. Install Android Studio. During setup, install Android SDK and set up an emulator (Pixel 6, API 33+).

73. In Android Studio, install the Flutter and Dart plugins (Settings > Plugins).
74. Run 'flutter doctor' in terminal. Resolve any issues shown (usually Android license acceptance).
75. Create a new Flutter project: 'flutter create fish_pond_app' in terminal.
76. Add the following dependencies to pubspec.yaml:

```
dependencies:  
  flutter:  
    sdk: flutter  
  firebase_core: ^2.24.0  
  firebase_auth: ^4.15.0  
  firebase_database: ^10.3.7  
  firebase_messaging: ^14.7.10  
  fl_chart: ^0.66.0          # For graphs  
  provider: ^6.1.1          # State management  
  intl: ^0.19.0             # Date/time formatting  
  shared_preferences: ^2.2.2 # Local settings storage
```

77. Run 'flutter pub get' to install dependencies.
78. Follow the FlutterFire documentation (firebase.flutter.dev) to connect your Firebase project: run 'flutterfire configure' and select your Firebase project.

6.3 App Architecture

The app uses the Provider pattern for state management, which is beginner-friendly yet scalable.

```
lib/  
├── main.dart                # App entry, Firebase init  
├── providers/  
│   ├── sensor_provider.dart # Listens to Firebase sensor data  
│   ├── device_provider.dart # Aerator/feeder state & commands  
│   └── settings_provider.dart # Threshold & schedule management  
├── screens/  
│   ├── dashboard_screen.dart # Home dashboard  
│   ├── aerator_screen.dart   # Aerator control & schedule  
│   ├── feeder_screen.dart    # Feeder control & schedule  
│   ├── history_screen.dart   # Historical data graphs  
│   ├── settings_screen.dart  # Threshold configuration  
│   └── alerts_screen.dart     # Alert log  
├── widgets/  
│   ├── sensor_card.dart      # Reusable sensor display card  
│   ├── status_card.dart      # Aerator/feeder status card  
│   └── schedule_tile.dart     # Schedule list item widget  
└── services/  
    ├── firebase_service.dart # All Firebase read/write operations  
    └── notification_service.dart # FCM notification handling
```

6.4 Key App Code Snippets

main.dart - Firebase Initialization

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:provider/provider.dart';
import 'firebase_options.dart';
import 'providers/sensor_provider.dart';
import 'providers/device_provider.dart';
import 'providers/settings_provider.dart';
import 'screens/dashboard_screen.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const FishPondApp());
}

class FishPondApp extends StatelessWidget {
  const FishPondApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => SensorProvider()),
        ChangeNotifierProvider(create: (_) => DeviceProvider()),
        ChangeNotifierProvider(create: (_) => SettingsProvider()),
      ],
      child: MaterialApp(
        title: 'Fish Pond Monitor',
        theme: ThemeData(
          colorScheme: ColorScheme.fromSeed(
            seedColor: const Color(0xFF1B4F8C)),
          useMaterial3: true,
        ),
        home: const DashboardScreen(),
      ),
    );
  }
}
```

sensor_provider.dart - Real-time Firebase Listener

```
import 'package:flutter/material.dart';
import 'package:firebase_database/firebase_database.dart';

class SensorProvider extends ChangeNotifier {
  double temperature = 0.0;
  double phValue = 0.0;
  int turbidity = 0;
  String timestamp = '--:--:--';
```

```
SensorProvider() { _listenToSensors(); }

void _listenToSensors() {
  FirebaseDatabase.instance.ref('/sensors').onValue.listen((event) {
    final data = event.snapshot.value as Map<dynamic, dynamic>;
    if (data == null) return;
    temperature = (data['temperature'] as num).toDouble();
    pHValue      = (data['ph'] as num).toDouble();
    turbidity    = (data['turbidity'] as num).toInt();
    timestamp    = data['timestamp']?.toString() ?? '--';
    notifyListeners();
  });
}
```

device_provider.dart - Sending Commands to ESP32

```
import 'package:flutter/material.dart';
import 'package:firebase_database/firebase_database.dart';

class DeviceProvider extends ChangeNotifier {
  String aeratorStatus = 'OFF';
  String feederStatus  = 'OFF';
  String lastFeedTime  = '--';

  DeviceProvider() { _listenToDevice(); }

  void _listenToDevice() {
    FirebaseDatabase.instance.ref('/device').onValue.listen((event) {
      final data = event.snapshot.value as Map<dynamic, dynamic>;
      if (data == null) return;
      aeratorStatus = data['aerator_status']?.toString() ?? 'OFF';
      lastFeedTime  = data['last_feed_time']?.toString() ?? '--';
      notifyListeners();
    });
  }

  // Send manual run command to ESP32
  Future<void> runAerator(int durationSeconds) async {
    await FirebaseDatabase.instance
      .ref('/commands/aerator')
      .set('RUN:$durationSeconds');
  }

  Future<void> runFeeder(int durationSeconds) async {
    await FirebaseDatabase.instance
      .ref('/commands/feeder')
      .set('RUN:$durationSeconds');
  }
}
```

dashboard_screen.dart - Home Dashboard

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/sensor_provider.dart';
import '../providers/device_provider.dart';
import '../widgets/sensor_card.dart';

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final sensors = context.watch<SensorProvider>();
    final device = context.watch<DeviceProvider>();

    return Scaffold(
      appBar: AppBar(
        title: const Text('Fish Pond Monitor'),
        backgroundColor: const Color(0xFF1B4F8C),
        foregroundColor: Colors.white,
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Text('Last updated: ${sensors.timestamp}',
              style: const TextStyle(color: Colors.grey)),
            const SizedBox(height: 12),
            SensorCard(
              label: 'Temperature',
              value: '${sensors.temperature.toStringAsFixed(1)} °C',
              icon: Icons.thermostat,
              status: _tempStatus(sensors.temperature),
            ),
            SensorCard(
              label: 'pH Level',
              value: sensors.phValue.toStringAsFixed(2),
              icon: Icons.science,
              status: _phStatus(sensors.phValue),
            ),
            SensorCard(
              label: 'Turbidity',
              value: '${sensors.turbidity} (raw)',
              icon: Icons.water,
              status: sensors.turbidity < 400 ? 'normal' : 'alert',
            ),
            const SizedBox(height: 16),
            // Quick manual controls
            Row(children: [
              Expanded(
                child: ElevatedButton.icon(
                  onPressed: () =>
                    device.runAerator(60), // 60 seconds
```

```

        icon: const Icon(Icons.air),
        label: const Text('Run Aerator (60s)'),
      ),
    ),
    const SizedBox(width: 12),
    Expanded(
      child: ElevatedButton.icon(
        onPressed: () =>
          device.runFeeder(5), // 5 seconds
        icon: const Icon(Icons.set_meal),
        label: const Text('Feed Now (5s)'),
      ),
    ),
  ],
),
],
),
),
bottomNavigationBar: _buildBottomNav(context),
);
}

String _tempStatus(double t) {
  if (t > 32.0 || t < 20.0) return 'alert';
  if (t > 30.0 || t < 22.0) return 'warning';
  return 'normal';
}

String _phStatus(double ph) {
  if (ph > 9.0 || ph < 6.0) return 'alert';
  if (ph > 8.5 || ph < 6.5) return 'warning';
  return 'normal';
}

BottomNavigationBar _buildBottomNav(BuildContext context) {
  return BottomNavigationBar(
    type: BottomNavigationBarType.fixed,
    items: const [
      BottomNavigationBarItem(icon: Icon(Icons.dashboard), label: 'Dashboard'),
      BottomNavigationBarItem(icon: Icon(Icons.air), label: 'Aerator'),
      BottomNavigationBarItem(icon: Icon(Icons.set_meal), label: 'Feeder'),
      BottomNavigationBarItem(icon: Icon(Icons.show_chart), label: 'History'),
      BottomNavigationBarItem(icon: Icon(Icons.settings), label: 'Settings'),
    ],
    // Add navigation logic here
  );
}
}

```

sensor_card.dart - Reusable Status Card Widget

```

import 'package:flutter/material.dart';

class SensorCard extends StatelessWidget {

```

```
final String label, value, status;
final IconData icon;

const SensorCard({
  super.key,
  required this.label,
  required this.value,
  required this.icon,
  required this.status,
});

Color get _color {
  switch (status) {
    case 'alert': return const Color(0xFFC62828);
    case 'warning': return const Color(0xFFF57F17);
    default: return const Color(0xFF2E7D32);
  }
}

@override
Widget build(BuildContext context) {
  return Card(
    margin: const EdgeInsets.only(bottom: 12),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
      side: BorderSide(color: _color, width: 2),
    ),
    child: ListTile(
      leading: CircleAvatar(
        backgroundColor: _color,
        child: Icon(icon, color: Colors.white),
      ),
      title: Text(label,
        style: const TextStyle(fontWeight: FontWeight.bold)),
      trailing: Text(value,
        style: TextStyle(
          fontSize: 20,
          fontWeight: FontWeight.bold,
          color: _color,
        )
      ),
    ),
  );
}
```

6.5 Building and Running the App

79. Connect an Android device via USB with USB Debugging enabled (Developer Options).
80. Run 'flutter run' in the project directory. The app will build and install on the device.
81. For a release APK: run 'flutter build apk --release'. The APK will be at build/app/outputs/flutter-apk/app-release.apk. Share this file for installation.
82. For iOS (requires a Mac): run 'flutter build ios --release' and archive via Xcode.

7. Full System Configuration & Integration

7.1 First Boot Checklist

Complete these steps in order on first deployment:

83. Flash firmware to ESP32 with correct WIFI_SSID, WIFI_PASSWORD, and Firebase credentials in config.h.
84. Power on ESP32. Open Serial Monitor. Confirm 'Connected! IP: xxx.xxx.xxx.xxx' message.
85. In Firebase Console, navigate to Realtime Database. Confirm sensor values are updating every 15 seconds.
86. Open the mobile app. Verify the dashboard shows live sensor values matching Serial Monitor output.
87. Test manual aerator: tap 'Run Aerator (60s)' in app. Confirm relay click and aerator starts within 15 seconds (next Firebase poll cycle).
88. For faster command response: implement Firebase real-time listener on ESP32 using Firebase.RTDB.set and stream callback (see Firebase-ESP-Client library documentation for streaming).
89. Test manual feeder: tap 'Feed Now (5s)'. Confirm feeder motor runs for 5 seconds.
90. Simulate a threshold breach: heat the water slightly above 32°C or bring a finger near the temp sensor probe. Verify buzzer, LED, and push notification all trigger.
91. Test a feeder schedule: set a schedule for 2 minutes from the current time. Confirm feeder activates at the correct time.

7.2 Improving Command Response Time

The polling-based command check (every 15 seconds) creates a noticeable delay for manual commands. For near-instant response, upgrade the firmware to use Firebase streaming:

```
// Add to firmware for real-time command streaming
// Replace checkCommands() polling with this callback approach

void streamCallback(FirebaseStream data) {
  if (data.dataPath() == "/commands/aerator") {
    String cmd = data.stringData();
    if (cmd.startsWith("RUN:")) {
      int secs = cmd.substring(4).toInt();
      setAerator(true, secs * 1000UL);
      Firebase.setString(fbData, "/commands/aerator", "IDLE");
    }
  }
  if (data.dataPath() == "/commands/feeder") {
    String cmd = data.stringData();
    if (cmd.startsWith("RUN:")) {
```



```
        int secs = cmd.substring(4).toInt();
        setFeeder(true, secs * 1000UL);
        Firebase.setString(fbData, "/commands/feeder", "IDLE");
    }
}

void streamTimeoutCallback(bool timeout) {
    if (timeout) Firebase.beginStream(fbData, "/commands");
}

// In setup(), after Firebase.begin():
// Firebase.beginStream(fbData, "/commands");
// Firebase.setStreamCallback(fbData, streamCallback, streamTimeoutCallback);
```

7.3 Threshold & Schedule Configuration via App

The Settings screen in the app should write threshold values directly to Firebase `/settings/` path. The ESP32 will pick them up on the next `loadSettings()` call. Example Flutter code for saving a threshold:

```
// In settings_screen.dart or settings_provider.dart
Future<void> saveThreshold({
    required String key,
    required double value,
}) async {
    await FirebaseDatabase.instance
        .ref('/settings/$key')
        .set(value);
}

// Usage:
// saveThreshold(key: 'temp_high', value: 32.0);
// saveThreshold(key: 'ph_low', value: 6.5);

// For schedules:
Future<void> saveFeederSchedule(int index, String time, int durationS) async {
    await FirebaseDatabase.instance
        .ref('/schedules/feeder/$index')
        .set({'time': time, 'duration_s': durationS, 'enabled': true});
}
```

7.4 Buzzer and LED Alert Behavior

The alert logic is implemented in the firmware's threshold check section. Here is the intended behavior:

- Any single parameter exceeding its threshold triggers BOTH the buzzer and LED simultaneously.

- The alert stays active until the parameter returns to within the safe range.
- The 10-minute cooldown (ALERT_COOLDOWN) prevents repeated push notifications for a sustained breach, but the local buzzer/LED remains active the entire time the condition persists.
- To add an 'Acknowledge Alert' feature: write a flag to Firebase /commands/acknowledge. The firmware reads this, silences the buzzer and LED, and waits until the next alert condition.

```
// Add to firmware loop() inside threshold check:
// Check for acknowledgement from app
if (Firebase.getString(fbData, "/commands/acknowledge")) {
  if (fbData.stringData() == "YES") {
    triggerAlert(false); // Silence local alert
    Firebase.setString(fbData, "/commands/acknowledge", "NO");
  }
}
```

8. Testing & Validation Plan

8.1 Unit Tests

92. Temperature sensor: Dip probe in ice water (~0°C) and boiling water (~100°C). Verify serial output matches.
93. pH sensor: Dip in pH 4.0 buffer: verify reading 3.8-4.2. Dip in pH 7.0 buffer: verify 6.8-7.2.
94. Turbidity: In clear distilled water: low reading. In muddy water: high reading. Verify values change appropriately.
95. Relay 1 (Aerator): Send 'RUN:10' command from app. Verify relay clicks ON, aerator starts, and stops after 10 seconds.
96. Relay 2 (Feeder): Send 'RUN:5' command. Verify motor runs for exactly 5 seconds.
97. Buzzer and LED: Set temp_high threshold to 25°C in app settings. Hold temp sensor in hand (~36°C). Verify buzzer and LED activate.

8.2 Integration Tests

98. Full data flow: Sensor reads data -> ESP32 processes -> Firebase updates -> App displays. Verify end-to-end latency is under 30 seconds.
99. Threshold breach -> push notification: Trigger a breach. Verify push notification arrives within 30 seconds.
100. Schedule test: Set feeder schedule for 2 minutes from now. Verify it fires at the correct time.
101. Offline resilience: Disconnect WiFi. Verify local alerts still work. Reconnect WiFi. Verify Firebase updates resume.

8.3 Common Issues & Fixes

Problem	Solution
DS18B20 reads -127°C	Pull-up resistor missing or wrong value. Verify 4.7k between data and 3.3V.
pH reads 0 or 14 always	Check ADC voltage divider. Verify GPIO 34 is receiving voltage in range 0-3.3V.
Relay clicks but aerator doesn't start	Check aerator is plugged into the NO terminal, not NC terminal. Check power supply.
Firebase connection timeout	Check FIREBASE_HOST format (no https://, no trailing slash). Verify Wi-Fi signal strength.
ESP32 resets repeatedly	Power issue. Use a USB adapter rated for 2A minimum. Relay inductive kick can cause resets - add 100uF capacitor on 5V rail.
Turbidity sensor reads max always	Check that ADC pin receives 0-3.3V. Apply voltage divider if module output exceeds 3.3V.
App not receiving push notifications	Verify FCM server key is correct. Check that firebase_messaging is initialized in main.dart. Ensure notification permissions are granted on device.
pH calibration drifts over time	Clean probe with pH electrode cleaning solution. Recalibrate monthly.

9. Recommended Next Steps & Improvements

- Implement Firebase real-time streaming in ESP32 firmware for instant command response (sub-second latency instead of 15-second polling).
- Store historical sensor data in Firebase (write to /history/YYYY-MM-DD/HH:MM:SS node). Implement data retention cleanup function to delete records older than 30 days.
- Add WiFi provisioning: Use ESP32 BLE provisioning or WiFiManager library so users can change Wi-Fi credentials from the app without reflashing.
- Add OTA (Over-the-Air) firmware updates using the ArduinoOTA library so future firmware updates can be pushed without physical USB connection.
- Feeder mechanical design: Use a 12V DC geared motor with a paddle or auger mechanism. 3D-print or fabricate a hopper that attaches to the pond edge. The motor rotation dispenses feed when the relay is energized.
- Add dissolved oxygen (DO) sensor (Atlas Scientific EZO-DO module, ~3000 BDT) for more complete water quality monitoring.
- Implement data export in the Flutter app: allow users to download sensor history as CSV via email or cloud share.
- Add local time display on ESP32 using an RTC module (DS3231, ~200 BDT) to ensure schedules work accurately even without internet connectivity.

End of Engineering Implementation Guide

IoT Fish Pond Management System | Engineering Team Document | Version 1.0